ТОО ”Колледж Хекслет”

Лабораторная работа №1

Подключение Internet Shop API к PostgreSQL

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Подключение Internet Shop API к PostgreSQL.

Цель: Создать базу данных PostgreSQL для интернет-магазина, подключить её к существующему Express-приложению через пакет pg и заменить чтение товаров из массива операциями SELECT.

Условие: RO4_3/lesson1/ЛР-1.pdf

Требования задания

• использовать PostgreSQL вместо массива для чтения товаров;

• содержать базу shop_db;

• содержать таблицу products;

• подключаться к PostgreSQL через пакет pg;

• использовать Pool;

• хранить параметры подключения в .env;

• иметь отдельный config/db.js;

• выполнять SELECT через pool.query();

• получать строки через result.rows;

• использовать параметр $1 при поиске по id;

• возвращать прежний формат REST API;

• возвращать 404 для отсутствующего товара;

• передавать ошибки PostgreSQL в существующий errorHandler;

• сохранять данные после перезапуска Node.js.

Ход выполнения

Создана таблица products с identity primary key, обязательными полями и CHECK(price > 0). pg Pool читает конфигурацию из .env. Чтение товаров выполняется параметризованными SELECT, in_stock преобразуется в inStock. Данные хранятся в PostgreSQL и не зависят от жизненного цикла Express.

Проект: RO4_3/lesson1

Запуск: npm install; npm start. Для PostgreSQL параметры подключения

задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

sql/01_products.sql

sql/01_products.sql

```sql
CREATE TABLE products (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    price INTEGER NOT NULL CHECK (price > 0),
    category VARCHAR(100) NOT NULL,
    in_stock BOOLEAN NOT NULL DEFAULT true
);
INSERT INTO products (name, price, category, in_stock) VALUES
('Keyboard', 15000, 'Accessories', true),
('Mouse', 8000, 'Accessories', true),
('Monitor', 85000, 'Monitors', false),
('Laptop', 420000, 'Computers', true),
('Webcam', 22000, 'Accessories', false);
```

Листинг 1. sql/01_products.sql

config/db.js

config/db.js

```js
import './env.js';
import { Pool } from 'pg';
export const pool = new Pool({
    ...(process.env.DATABASE_URL ? { connectionString: process.env.DATABASE_URL } : {
        host: process.env.PGHOST,
        port: Number(process.env.PGPORT || 5432),
        database: process.env.PGDATABASE,
        user: process.env.PGUSER,
        password: process.env.PGPASSWORD
    }),
    connectionTimeoutMillis: 3000
});
pool.on('error', error => console.error('PostgreSQL pool:', error.message));
```

Листинг 2. config/db.js

services/productService.js

services/productService.js

```
import { pool } from '../config/db.js';
export async function getAllProducts(query = {}) {
    const result = await pool.query(`SELECT p.id, p.name, p.price, p.category,
p.in_stock AS "inStock" FROM products p
        WHERE ($1::text IS NULL OR p.category = $1)
        AND ($2::boolean IS NULL OR p.in_stock = $2)
        AND ($3::integer IS NULL OR p.price >= $3)
        AND ($4::integer IS NULL OR p.price <= $4) ORDER BY p.id`,
        [query.category ?? null, query.inStock ?? null, query.minPrice ?? null,
query.maxPrice ?? null]);
    return result.rows;
}
export async function findProductById(id) {
    const result = await pool.query(`SELECT p.id, p.name, p.price, p.category,
p.in_stock AS "inStock" FROM products p WHERE p.id = $1`, [id]);
    return result.rows[0];
}
```

Листинг 3. services/productService.js

test-db.js

test-db.js

```
import { pool } from './config/db.js';
try { console.log('PostgreSQL connected:', (await pool.query('SELECT NOW() AS
current_time')).rows[0]); }
catch (error) { console.error(error.message); process.exitCode = 1; }
finally { await pool.end(); }
```

Листинг 4. test-db.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

Примеры ответов

GET /api/products → 200


```
[
  {
    "id": 1,
    "name": "Keyboard",
    "price": 15000,
    "category": "Accessories",
    "inStock": true
  },
  {
    "id": 2,
    "name": "Mouse",
    "price": 8000,
    "category": "Accessories",
    "inStock": true
  },
  {
    "id": 3,
    "name": "Monitor",
    "price": 85000,
```

```
    "category": "Monitors",
    "inStock": false
  },
  {
    "id": 4,
    "name": "Laptop",
    "price": 420000,
    "category": "Computers",
    "inStock": true
  },
  {
    "id": 5,
    "name": "Webcam",
    "price": 22000,
    "category": "Accessories",
    "inStock": false
  }
]
```


GET /api/products/1 → 200


```
{
  "id": 1,
  "name": "Keyboard",
  "price": 15000,
  "category": "Accessories",
  "inStock": true
}
```


Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.